

Práctica 6

Capa de Transporte - Parte 2

1. ¿Cuál es el puerto por defecto que se utiliza en los siguientes servicios? Web / SSH / DNS / Web Seguro / POP3 / IMAP / SMTP

Estos son los números universales estandarizados por la **IANA** (Internet Assigned Numbers Authority)

Servicio	Nombre Técnico	Puerto por defecto	Protocolo de Transporte usual
Web	HTTP	80	TCP
SSH	Secure Shell	22	TCP
DNS	Domain Name System	53	UDP (principalmente para consultas) / TCP (zonas)
Web Seguro	HTTPS	443	TCP
POP3	Post Office Protocol v3	110	TCP
IMAP	Internet Message Access Protocol	143	TCP
SMTP	Simple Mail Transfer Protocol	25	TCP

SMTP sirve para enviar mails, mientras que POP3 e IMAP sirven para recibirlos/descargarlos.

Investigue en qué lugar en Linux y en Windows está descrita la asociación utilizada por defecto para cada servicio.

Tanto en Linux como en Windows existe un archivo de texto plano idéntico en su estructura llamado `services`. Este archivo es el que consulta el sistema operativo (y comandos como `ss` o `netstat` cuando no usás la opción `-n`) para traducir un número de puerto a su nombre amigable

- En Linux el archivo se encuentra estrictamente en la ruta: `/etc/services`
- En Windows la ruta es `C:\Windows\System32\drivers\etc\services`

2. Investigue qué es multicast. ¿Sobre cuál de los protocolos de capa de transporte funciona? ¿Se podría adaptar para que funcione sobre el otro protocolo de capa de transporte? ¿Por qué?

- Multicast es un método de direccionamiento y transmisión de datos en redes donde un único emisor envía paquetes a un grupo específico de receptores interesados (un grupo multicast) en simultáneo.

A nivel de eficiencia, es brillante: el emisor genera un solo paquete y son los routers de la red los que se encargan de duplicarlo únicamente cuando los caminos se bifurcan para llegar a los miembros del grupo. diferencia de Broadcast (que le rompe las bolas a toda la red entera, esté interesada o no) y de Unicast (donde si tenés 100 clientes tenés que mandar 100 paquetes individuales), Multicast solo entrega el tráfico a quienes se suscribieron explícitamente al grupo.

- Funciona exclusivamente sobre UDP (User Datagram Protocol). Las direcciones IP de Multicast (Clase D, de la 224.0.0.0 a la 239.255.255.255) se mapean directamente con sockets UDP para que las aplicaciones escuchen esos datagramas en silencio.
- Es técnicamente imposible adaptar Multicast para que funcione sobre TCP. La razón fundamental es que TCP es un protocolo estrictamente orientado a la conexión y de punto a punto (Unicast). Si un emisor enviara un paquete TCP a un grupo de 1000 receptores, esos 1000 receptores le responderían con un paquete ACK en simultáneo al emisor para confirmar el recibo. El emisor sufriría una implosión de ACKs, saturando su placa de red y su procesador al intentar mantener 1000 estados de conexión diferentes y retransmisiones para un solo paquete enviado. TCP no está diseñado matemáticamente para hablarle a una masa de compus en un solo pipe.

3. Investigue cómo funciona el protocolo de aplicación FTP teniendo en cuenta las diferencias en su funcionamiento cuando se utiliza el modo activo de cuando se utiliza el modo pasivo ¿En qué se diferencian estos tipos de comunicaciones del resto de los protocolos de aplicación vistos?

A diferencia de HTTP (80), SSH (22) o SMTP (25), que meten todo su tráfico en una sola conexión, FTP utiliza dos conexiones TCP independientes y paralelas para funcionar:

- La conexión de Control (Puerto 21 del Servidor): Se usa exclusivamente para pasar los comandos de texto (ej. loguearse, listar carpetas, pedir un archivo). Está abierta durante toda la sesión.
- La conexión de Datos (Puerto dinámico): Se abre y se cierra cada vez que se transfiere un archivo real o se pide un listado de directorio.

Modo Activo (El Servidor conecta al Cliente)

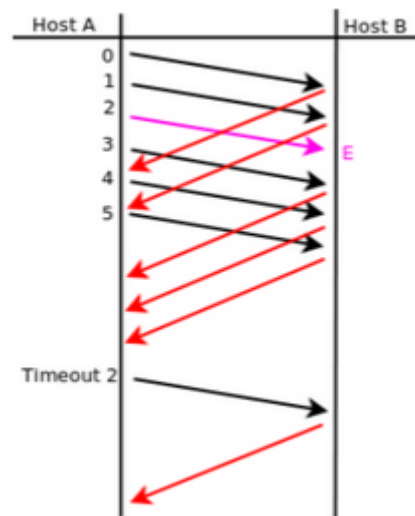
1. El cliente abre la conexión de control pegándole al puerto **21** del servidor.
2. Cuando el cliente quiere descargar un archivo, se pone a escuchar en un puerto aleatorio propio (ej. puerto **X**) y le manda un comando especial (**PORT**) al servidor diciéndole: *"Estoy listo, conectate a mi puerto X"*.
3. **El Servidor inicia activamente la conexión de datos** saliendo desde su puerto **20** directo hacia el puerto **X** del cliente.
- **El problema del Modo Activo:** Hoy en día casi no se usa porque los clientes suelen estar detrás de un **Firewall o NAT** (como el router). Cuando el servidor intenta meterse activamente a la PC del cliente en el puerto **X**, el firewall del cliente bloquea la conexión entrante por seguridad y la transferencia falla.

Modo Pasivo (El Cliente conecta al Servidor)

Se inventó justamente para solucionar el problema de los firewalls del lado del cliente.

1. El cliente abre la conexión de control en el puerto **21** del servidor.
2. Al querer transferir un archivo, el cliente le manda el comando **PASV**.
3. El servidor abre un puerto aleatorio alto en su propia máquina (ej. puerto **Y**) y le responde al cliente: *"Ponete en modo pasivo; yo abrí el puerto Y en mi máquina, conectate vos acá"*.
4. **El Cliente inicia la conexión de datos**, pegándole desde un puerto propio al puerto **Y** del servidor.

4. Suponiendo Selective Repeat; tamaño de ventana 4 y sabiendo que E indica que el mensaje llegó con errores. Indique en el siguiente gráfico, la numeración de los ACK que el host B envía al Host A.



¿Qué es Selective Repeat ?

Es un es un tipo de respuesta usado en control de errores

A diferencia de *Go-Back-N* (donde si un paquete se pierde, se descarta todo lo que viene atrás y se retransmite el bloque entero), en **Selective Repeat el receptor acepta y guarda en un búfer los paquetes que llegan desordenados pero sanos**. Solo le pide al emisor que retransmita **únicamente el paquete específico que faltaba o llegó con errores**.

Se envían paquetes hasta que se recibe un NACK o hasta que se completa la ventana de transmisión definida; en ese momento se termina de enviar el paquete que estábamos transmitiendo y se reenvía el paquete que tenía errores; inmediatamente después se sigue enviando la información a partir del último paquete que se había enviado.

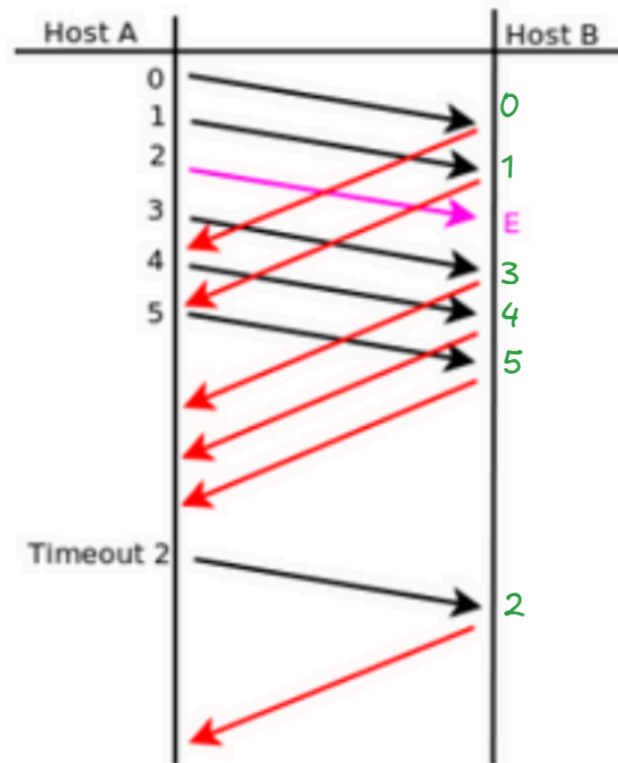
¿Qué significa que el tamaño de la ventana sea 4 (N=4)?

Significa que el emisor tiene permitido inyectar en la red un **máximo de 4 paquetes consecutivos sin haber recibido todavía un acuse de recibo (ACK)** del primero de ellos.

Mapeándolo con el gráfico: el Host A puede mandar los paquetes 0, 1, 2 y 3 juntos. Hasta que no le vuelva el ACK 0, su ventana se queda "trabada" ahí y no puede mandar el paquete 4. Cuando le llega el ACK 0, la ventana se desliza un lugar hacia la derecha y habilita el envío del 4

Respuesta:

En este protocolo, **los ACKs son individuales (selectivos)**. Cada ACK lleva estrictamente el número del paquete que se acaba de recibir correctamente. No son acumulativos.



Cuando llega el paquete 2 con errores (E): Host B detecta el error y DESCARTA el paquete. No envía ningún ACK para el paquete 2 (por eso la flecha rosa no genera ninguna flecha roja saliente).

Desde la perspectiva de A:

- Le llegan los ACK 0 y ACK 1 → Desliza la ventana.
- Pasa el tiempo y el temporizador exclusivo del paquete 2 se vence (Timeout 2).
- Al saltar el timeout, Host A retransmite únicamente el paquete 2 (La flecha negra solitaria de abajo).
- Cuando este paquete 2 llega sano a Host B, el Host B ya tiene en su búfer el 3, 4 y 5 ordenaditos. Así que acepta el 2 y devuelve el ACK 2 (Última flecha roja).

5. ¿Qué restricción existe sobre el tamaño de ventanas en el protocolo Selective Repeat?

En el protocolo Selective Repeat, el tamaño de la ventana de transmisión y el tamaño de la ventana de recepción (que generalmente son iguales) deben ser **menores o iguales a la mitad del espacio total de números de secuencia disponible**.

Si la ventana fuera más grande que la mitad del espacio de secuencia, el receptor no tendría forma de distinguir si un paquete que le acaba de llegar es un paquete nuevo o si es una retransmisión vieja que se había retrasado en la red (dado que la ventana es circular)

Al limitar el tamaño de la ventana a la mitad del espacio de secuencia, se garantiza matemáticamente que la ventana del emisor y la del receptor nunca se solapen de forma ambigua. Si se pierden todos los ACKs, el paquete retransmitido caerá fuera de la ventana nueva del receptor, permitiéndole identificarlo correctamente como un duplicado viejo y descartarlo.

6. De acuerdo a la captura TCP de la siguiente figura, indique los valores de los campos borroneados

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.1.1	172.20.1.100	TCP	74	41749 > vce [SYN] Seq=3933822137 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=270132 TSecr=0
2	0.001264	172.20.1.100	172.20.1.1	TCP	74	vce > 41749 [SYN, ACK] Seq=1047471501 Ack=3933822138 Win=5792 Len=0 MSS=1460 SACK_PERM=1
3	0.001341	172.20.1.1	172.20.1.100	TCP	66	41749 > VCE [ACK] Seq=3933822138 Ack=1047471502 Win=5888 Len=0 TSval=270132 TSecr=1877442

Internet Protocol Version 4, Src: 172.20.1.100 (172.20.1.100), Dst: 172.20.1.1 (172.20.1.1)

Transmission Control Protocol, Src Port: vce (11111), Dst Port: 41749 (41749), Seq: 1047471501, Ack: 3933822138, Len: 0

Source port: vce (11111)
Destination port: 41749 (41749)
[Stream index: 0]
Sequence number: 1047471501
Acknowledgement number: 3933822138
Header length: 40 bytes

Flags: 0x012 (SYN, ACK)

000. = Reserved: Not set
...0 = Nonce: Not set
.... 0... = Congestion Window Reduced (CWR): Not set
.... .0.. = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...1 = Acknowledgement: Set
....0 = Push: Not set
....0.. = Reset: Not set
....1. = Syn: Set
....0 = Fin: Not set
Window size value: 5792
[Calculated window size: 5792]
Checksum: 0x9803 [validation disabled]

Explicación:

Línea 1:

- **SYN:** Lo sabemos porque es el primer paso del 3-Way Handshake
- **Seq=3933822137:** Lo sabemos mirando la línea 2, que en el segundo paso le respondieron ese num+1, lo que nos da la pauta para completar esa línea.

Línea 3:

- **172.20.1.1:** Como es el 3er paso del 3WHS, sacamos esta IP de la línea 1.
- **172.20.1.100:** Ídem anterior
- **41749 > VCE:** Ídem anterior
- **Seq=3933822138:** El cliente en el paquete 1 mandó su Seq=3933822137. En su siguiente paquete (este, el 3), su número de secuencia avanza a 3933822138
- **Ack=1047471502:** El cliente tiene que acusar recibo del SYN del servidor. Como el servidor mandó Seq=1047471501 en el paso 2, el cliente le responde con ese número más 1.

7. Dada la sesión TCP de la figura, completar los valores marcados con un signo de interrogación.

Reglas para resolverlo:

- Los flags SYN y FIN consumen 1 byte virtual de secuencia.
- Los datos reales consumen tantos bytes de secuencia como diga el campo Len.
- Los paquetes que son puramente un ACK (sin datos, Len=0) NO consumen números de secuencia. El próximo paquete con datos va a usar ese mismo número.

Time	10.0.0.10	10.0.1.10	Comment
1.360	(54762)	→ SYN → (10000)	Seq = 0
1.360	(54762)	← SYN, ACK → (10000)	Seq = 0 Ack = 1
1.360	(54762)	→ ACK → (10000)	Seq = 1 Ack = 1
3.581	(54762)	→ PSH, ACK - Len: 7 → (10000)	Seq = 1 Ack = 1
3.581	(54762)	← ACK → (10000)	Seq = 1 Ack = 8
8.796	(54762)	→ PSH, ACK - Len: 9 → (10000)	Seq = 8 Ack = 1
8.797	(54762)	← ACK → (10000)	Seq = 1 Ack = 17
14.382	(54762)	→ PSH, ACK - Len: 5 → (10000)	Seq = 17 Ack = 1
14.382	(54762)	← ACK → (10000)	Seq = 1 Ack = 22
15.190	(54762)	→ FIN, ACK → (10000)	Seq = 22 Ack = 1
15.190	(54762)	← FIN, ACK → (10000)	Seq = 1 Ack = 23
15.190	(54762)	→ ACK → (10000)	Seq = 23 Ack = 2

8. ¿Qué es el RTT y cómo se calcula? Investigue la opción TCP timestamp y los campos TSval y TSecr.

El RTT (Round trip-time, Tiempo de Ida y Vuelta) es el tiempo que tarda un segmento de datos en viajar desde el emisor hasta el receptor, sumado al tiempo que tarda en volver el ACK correspondiente al emisor. Básicamente, es el tiempo total del "viaje redondo" de la información por la red.

Se incluye un TCP Timestamp en el encabezado.

Esto permite incluir dos campos de tiempo de 4 bytes en cada segmento TCP: TSval y TSecr.

- **TSval (Timestamp Value):**

Es el sello de tiempo que estampa el emisor en el paquete.

Contiene el valor actual del reloj interno de su propia máquina en el momento exacto en que el paquete sale al cable.

- **TSecr (Timestamp Echo Reply):**

Es el "eco" del sello de tiempo. Este campo solo es válido si el paquete lleva el flag ACK encendido. Sirve para devolverle al otro host el último valor de TSval que se recibió de su parte.

El cálculo es una sencilla resta:

- El Cliente manda un paquete con sus datos y clava su reloj en el campo: TSval = 100 (por dar un número redondo). El campo TSecr va en 0 o con el tiempo del servidor si ya hablaron antes.
- El paquete viaja por la red, llega al Servidor, la aplicación procesa los datos y prepara el ACK de respuesta. Al armar el ACK, el Servidor mira su propio reloj interno (que va por el número 5000) y lo mete en su TSval = 5000. Pero además, copia el TSval que le mandó el cliente y lo pega en el campo TSecr. Es decir, devuelve: TSecr = 100.
- El paquete de vuelta llega al Cliente. En ese instante exacto, el Cliente mira su reloj interno actual y ve que marca el número 120. El Cliente toma el paquete, abre el campo TSecr (que dice 100) y hace una resta simple con su reloj actual:

$$\text{RTT} = \text{Reloj actual} - \text{TSecr}$$

9. Para la captura tcp-captura.pcap, responder las siguientes preguntas.

a. ¿Cuántos intentos de conexiones TCP hay?

tcp.flags.syn == 1						
No.	Time	Source	Destination	Protocol	Length	Info
3	0.000079	10.0.2.10	10.0.4.10	TCP	74	46907 → 5001 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SAC
4	0.000116	10.0.4.10	10.0.2.10	TCP	74	5001 → 46907 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M
961	82.420045	10.0.2.10	10.0.4.10	TCP	74	45670 → 7002 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SAC
963	83.540758	10.0.2.10	10.0.4.10	TCP	74	45671 → 7002 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SAC
967	97.968958	10.0.2.10	10.0.4.10	TCP	74	46910 → 5001 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SAC
968	97.969023	10.0.4.10	10.0.2.10	TCP	74	5001 → 46910 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M
981	135.753852	10.0.2.10	10.0.4.10	TCP	74	54424 → 9000 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SAC
982	135.754058	10.0.4.10	10.0.2.10	TCP	74	9000 → 54424 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M
1106	149.807117	10.0.2.10	10.0.4.10	TCP	74	54425 → 9000 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SAC
1107	149.807136	10.0.4.10	10.0.2.10	TCP	74	9000 → 54425 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 M

Hay 6 intentos de conexión, son aquellos que tienen solo la flag SYN.

b. ¿Cuáles son la fuente y el destino (IP:port) para c/u?

Intento 1 (Fila 3): 10.0.2.10:46907 → 10.0.4.10:5001

Intento 2 (Fila 961): 10.0.2.10:45670 → 10.0.4.10:7002

Intento 3 (Fila 963): 10.0.2.10:45671 → 10.0.4.10:7002

Intento 4 (Fila 967): 10.0.2.10:46910 → 10.0.4.10:5001

Intento 5 (Fila 981): 10.0.2.10:54424 → 10.0.4.10:9000

Intento 6 (Fila 1106): 10.0.2.10:54425 → 10.0.4.10:9000

c. ¿Cuántas conexiones TCP exitosas hay en la captura? ¿Cómo diferencia las exitosas de las que no lo son? ¿Cuáles flags encuentra en cada una?

Hay 4 conexiones exitosas.

¿Cómo se diferencian?

- **Exitosas:**

Son aquellas donde el servidor **respondió** al intento de conexión.

En la captura pantalla se ve porque inmediatamente abajo del paquete [SYN]

aparece un paquete en dirección contraria (10.0.4.10 hacia 10.0.2.10) con los flags [SYN, ACK] (es decir, el destino ACEPTÓ la conexión, 2do paso de 3WHS)

- **No exitosas (Fallidas):**

Son aquellas donde el cliente mandó el [SYN] pero no hubo respuesta [SYN,

ACK] del servidor (puede volver un RST+ACK o directamente no haber respuesta).

d. Dada la primera conexión exitosa responder:

i. ¿Quién inicia la conexión?

La inicia el host 10.0.2.10 (el cliente), enviando el segmento SYN en la fila 3.

ii. ¿Quién es el servidor y quién el cliente?

Cliente: 10.0.2.10 (puerto origen 46907)

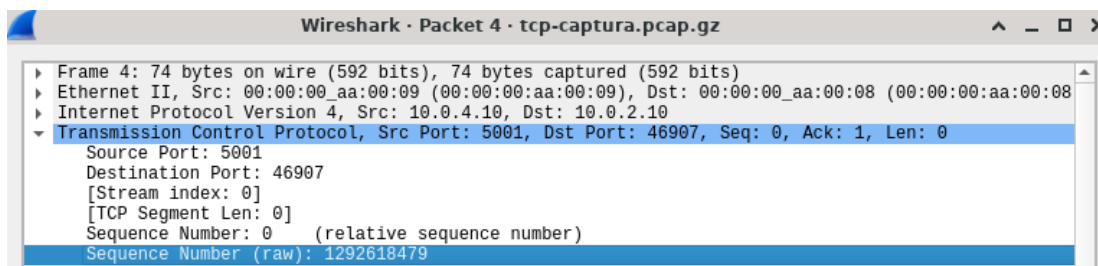
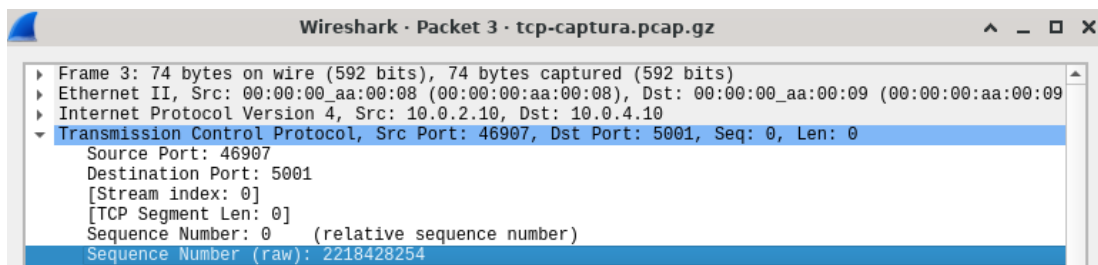
Servidor: 10.0.4.10 (puerto destino 5001)

iii. ¿En qué segmentos se ve el 3-way handshake?

Se ve claramente en las filas

- 3 (SYN)
- 4 (SYN, ACK)
- 5 (ACK).

iv. ¿Cuáles ISNs se intercambian?



v. ¿Cuál MSS se negoció?

MSS = Maximun segment size.

Ambos propusieron 1460.

tcp					
No.	Time	Source	Destination	Protocol	Length Info
3	0.000079	10.0.2.10	10.0.4.10	TCP	74 46907 → 5001 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM=1
4	0.000116	10.0.4.10	10.0.2.10	TCP	74 5001 → 46907 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460

vi. ¿Cuál de los dos hosts envía la mayor cantidad de datos (IP:port)?

El host que envía la mayor cantidad de datos es el cliente 10.0.2.10:46907.

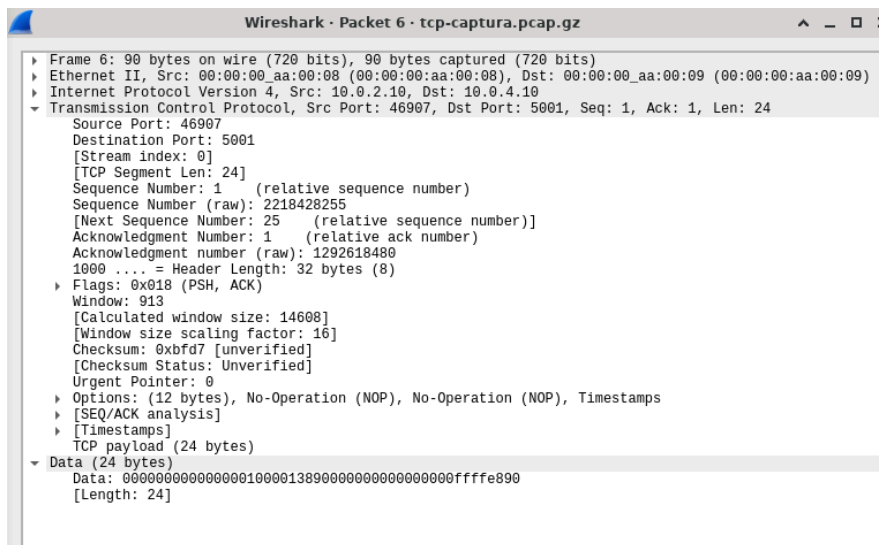
Se observa en la captura que el número de secuencia relativo del cliente aumenta de forma sostenida y drástica a lo largo del tiempo (alcanzando Seq=23193 hacia la fila 40 debido al envío continuo de segmentos con Len=1448 o Len=1460), lo que refleja está enviando constantemente de datos a la red.

En contraposición, el número de secuencia del servidor (10.0.4.10:5001) permanece estrictamente estancado en Seq=1 en toda la ráfaga, evidenciando que sus segmentos son únicamente ACK puros con Len=0 y no contienen datos de aplicación.

e. Identificar primer segmento de datos (origen, destino, tiempo, número de fila y número de secuencia TCP).

El primer paquete de datos es el frame 6, que le sigue instantaneamente después al 3er paso del 3WHS.

- Origen: 10.0.2.10
- Destino: 10.0.4.10
- Tiempo: 0.151826
- Numero de fila: 6
- Numero de secuencia TCP: 1



i. ¿Cuántos datos lleva?

Lleva 24 bytes de datos.

ii. ¿Cuándo es confirmado (tiempo, número de fila y número de secuencia TCP)?

Es confirmado en:

- Tiempo: 0.151925
 - Número fila: 7
 - Número de secuencia: 1
- (Dice ACK=25, porque es 1 (el numero de secuencia anterior) + 24 (bytes de datos))

iii. La confirmación, ¿qué cantidad de bytes confirma?

Confirma 24 bytes, ya que le está pidiendo el byte 25.

f. ¿Quién inicia el cierre de la conexión? ¿Qué flags se utilizan? ¿En cuáles segmentos se ve (tiempo, número de fila y número de secuencia TCP)?

- El cierre lo inicia el cliente 10.0.2.10:46907 en la fila 958.
- Se utilizan los flags FIN, PSH y ACK combinados en el primer paso.

Se ve en los siguientes segmentos:

958	75.090196	10.0.2.10	10.0.4.10	TCP	234	46907 → 5001	[FIN, PSH, ACK]	Seq=786289	Ack=1
959	75.091719	10.0.4.10	10.0.2.10	TCP	66	5001 → 46907	[FIN, ACK]	Seq=1	Ack=786458
960	75.247457	10.0.2.10	10.0.4.10	TCP	66	46907 → 5001	[ACK]	Seq=786458	Ack=2

Donde:

- **Fila 958:**
El cliente envía un segmento [FIN, PSH, ACK] con Seq=786289 y Ack=1, avisando que no tiene más datos para mandar y quiere cerrar su parte del canal.
- **Fila 959:**
El servidor (10.0.4.10) responde de manera inmediata con un segmento [FIN, ACK]. Al meter el FIN en este mismo paquete, el servidor le dice al cliente: "Recibí tu cierre (por eso Ack=786458) y yo también quiero cerrar mi lado ya mismo".
- **Fila 960:**
El cliente responde con el [ACK] final (Seq=786458, Ack=2), confirmando el cierre del servidor. La conexión queda formalmente liquidada.

Nota: El flag PSH (Push) sirve para decirle a la pila TCP: "No acumules más esto en el búfer, no esperes a que se llene, mandámelo inmediatamente"

10. Responda las siguientes preguntas respecto del mecanismo de control de flujo.

a. ¿Quién lo activa? ¿De qué forma lo hace?

Lo activa el Receptor de la conexión. Lo hace de forma constante y dinámica en cada paquete que envía de vuelta, notificando el tamaño de su búfer disponible en el campo Window (Ventana de Recepción / RcvWindow) del encabezado TCP

b. ¿Qué problema resuelve?

Resuelve el problema de que un emisor rápido sature y desborde el búfer del receptor. Si el emisor manda datos a más velocidad de la que la aplicación del receptor puede procesarlos y sacarlos de la memoria, el búfer se llena. El control de flujo evita que se pierdan paquetes por este desborde, obligando al emisor a disminuir la cantidad o a frenar

c. ¿Cuánto tiempo dura activo y qué situación lo desactiva?

Está activo durante toda la vida de la conexión TCP.

Monitorea el flujo en tiempo real de la siguiente manera:

- **Se extrema (frena al emisor):** Si el receptor se queda sin espacio, manda un paquete con Window = 0 (Ventana Cero). El emisor se congela y no puede mandar un solo byte más.
- **Se reanuda el envío:** Cuando la aplicación del receptor procesa los datos pendientes y libera espacio en el búfer, el receptor envía un paquete de actualización de ventana (Window Update) con un valor de Window > 0. Al recibir esto, el emisor vuelve a transmitir.

En todo momento ambos extremos están actualizando su propia ventana.

11. Responda las siguientes preguntas respecto del mecanismo de control de congestión.

a. ¿Quién activa el mecanismo de control de congestión? ¿Cuáles son los posibles disparadores?

Lo activa de forma autónoma el Emisor. Como la red (los routers intermedios) no le avisa formalmente si está saturada, el emisor lo deduce a partir de dos posibles disparadores (eventos de pérdida):

- **Triple ACK duplicado:** Si le llegan 3 ACKs idénticos seguidos pidiendo el mismo paquete asume que la red está congestionada pero "todavía se mueve" (algunos paquetes pasan).
- **Timeout (Vencimiento del temporizador):** No llega ningún ACK. Esto es el peor escenario: la red está completamente tapada y los paquetes directamente se caen.

b. ¿Qué problema resuelve?

Resuelve el problema de la saturación de los nodos intermedios de la red (routers y switches). Si muchos hosts mandan tráfico a la vez, los búfers de los routers de internet colapsan y empiezan a tirar paquetes a la basura. El control de congestión evita el colapso global de la red regulando la cantidad de datos que el emisor inyecta basándose en el estado del camino

c. Diferencie slow start de congestion-avoidance.

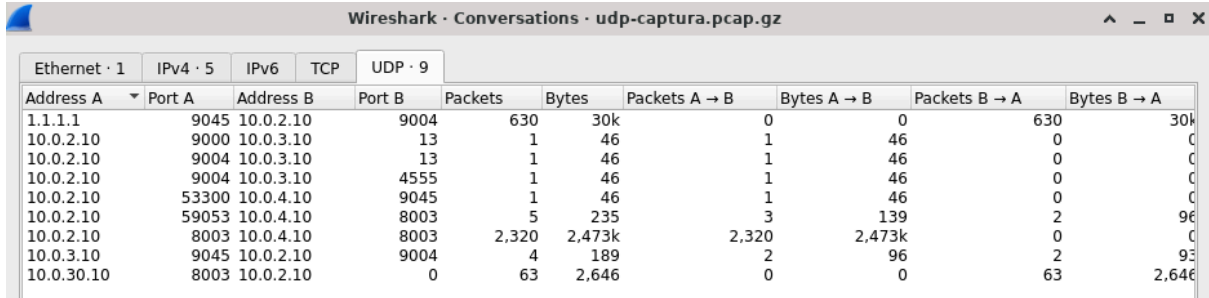
Ambos son algoritmos que manejan la Ventana de Congestión (Cwnd) del emisor, pero la modifican a ritmos totalmente distintos:

- **Slow Start (Arranque Lento):**
Se usa al principio de la conexión o después de un Timeout. Su crecimiento es exponencial. Arranca con un tamaño de ventana muy chico (ej. 1 MSS) y se duplica por cada RTT que pasa sin pérdidas. Su objetivo es tantear la red rápidamente para ver cuánto ancho de banda hay disponible.
- **Congestion Avoidance (Evitación de Congestión):**
Se activa cuando la ventana alcanza un límite seguro llamado umbral de arranque lento (Ssthresh). Acá el crecimiento pasa a ser lineal. En lugar de duplicarse, la ventana aumenta solo 1 MSS por cada RTT. Su objetivo es aproximarse al límite máximo de la red de forma muy cautelosa y milimétrica para no generar congestión.

12. Para la captura udp-captura.pcap, responder las siguientes preguntas.

a. ¿Cuántas comunicaciones (srcIP,srcPort,dstIP,dstPort) UDP hay en la captura?

Existen 9 comunicaciones UDP en total (filtré udp→ statics→ conversations→ pestaña udp)



Wireshark · Conversations · udp-captura.pcap.gz											
Ethernet · 1		IPv4 · 5		IPv6		TCP		UDP · 9			
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A		
1.1.1.1	9045	10.0.2.10	9004	630	30k	0	0	630	30k		
10.0.2.10	9000	10.0.3.10	13	1	46	1	46	0	0		
10.0.2.10	9004	10.0.3.10	13	1	46	1	46	0	0		
10.0.2.10	9004	10.0.3.10	4555	1	46	1	46	0	0		
10.0.2.10	53300	10.0.4.10	9045	1	46	1	46	0	0		
10.0.2.10	59053	10.0.4.10	8003	5	235	3	139	2	96		
10.0.2.10	8003	10.0.4.10	8003	2,320	2,473k	2,320	2,473k	0	0		
10.0.3.10	9045	10.0.2.10	9004	4	189	2	96	2	96		
10.0.30.10	8003	10.0.2.10	0	63	2,646	0	0	63	2,646		

b. ¿Cómo se podrían identificar las exitosas de las que no lo son?

En UDP no existe el concepto de "conexión exitosa" a nivel de capa de transporte porque UDP no tiene saludo de tres vías (handshake), ni acuses de recibo (ACK), ni control de flujo. UDP tira el paquete y se descarta, nadie se hace cargo.

Sin embargo, a nivel práctico en una captura, podés intuir si "fluyó" la comunicación de dos formas:

- Exitosas (Con respuesta): Si vemos que después de un datagrama en un sentido, el destino responde con otro datagrama UDP de vuelta (comunicación bidireccional, como una consulta DNS y su respuesta), podríamos estar bastante seguros de que fue exitosa.
- No exitosas (Fallidas): Si un host manda un datagrama UDP y el servidor le responde con un paquete ICMP (Destination unreachable - Port unreachable). Eso significa que el paquete llegó, pero el puerto UDP estaba cerrado y la comunicación murió ahí.

En otro caso, no podríamos asegurar nada (podría haber llegado o no).

c. ¿UDP puede utilizar el modelo cliente/servidor?

Sí, totalmente. Que el protocolo sea simple no quita que una máquina pueda tomar el rol de Servidor (quedándose escuchando en un puerto UDP fijo) y otra el de Cliente (enviando una solicitud a ese puerto).

d. ¿Qué servicios o aplicaciones suelen utilizar este protocolo? ¿Qué requerimientos tienen?

- Aplicaciones típicas: DNS (puerto 53), DHCP, streaming de video/audio en vivo (como Twitch), llamadas (Google Meet, Discord), juegos en línea y protocolos de consulta rápida como SNMP.
- Estas aplicaciones priorizan la velocidad y la baja latencia (tiempo de respuesta inmediato) por encima de la confiabilidad. Prefieren perder un fotograma de video o un milisegundo de audio antes que frenar toda la transmisión para retransmitir un paquete viejo (lo que generaría lag o congelamiento).
Por eso en juegos cuando hay pérdida de paquetes te teletransportas: no tiene sentido retransmitir lo que se perdió. Ídem en videollamadas, que al recuperar la conexión solo te traen al momento actual

e. ¿Qué hace el protocolo UDP en relación al control de errores?

UDP hace el mínimo esfuerzo posible: solo detecta errores, pero no los corrige.

- UDP incluye un campo opcional llamado Checksum en su cabecera. El receptor calcula este Checksum al recibir el datagrama.
- Si el Checksum da mal (el paquete se corrompió en el medio): El receptor tira el paquete a la basura de forma silenciosa y no le avisa a nadie. No hay retransmisiones automáticas. Si la aplicación necesita recuperar ese dato, se tiene que encargar el software de capa superior, pero UDP no se hace cargo.

f. Con respecto a los puertos vistos en las capturas, ¿observa algo particular que lo diferencie de TCP?

- **Sí. Lo particular que se observa en la captura UDP es la multiplexación libre de puertos sin mantener un estado de conexión único.**

A diferencia de TCP (donde un puerto local queda "bloqueado" de forma exclusiva a una conexión dedicada IP_origen:puerto_origen<->IP_destino:puerto_destino), en UDP un mismo puerto local puede enviar y recibir datagramas de múltiples destinos y procesos diferentes en simultáneo.

Por ejemplo, el puerto 9004 del host 10.0.2.10 interactúa de forma independiente con la IP 1.1.1.1 y con distintos puertos de la IP 10.0.3.10.

- **Además, podemos ver que el puerto de origen en UDP es totalmente opcional.**

En TCP, el puerto de origen es obligatorio porque, al ser orientado a la conexión, el receptor necesita saber a qué puerto exacto responder para mantener vivo el circuito.

Pero en UDP, si a la aplicación que manda los paquetes no le interesa en absoluto recibir una respuesta (por ejemplo, porque solo está inyectando telemetría, enviando un flujo unidireccional de datos, o haciendo una prueba de carga a un servidor), el protocolo permite rellenar el campo del puerto de origen con ceros binarios.

g. Dada la primera comunicación en la cual se ven datos en ambos sentidos (identificar el primer datagrama):

79	59.092837	10.0.2.10	10.0.3.10	UDP	46	9004 → 9045	Len=4
80	62.173832	10.0.3.10	10.0.2.10	UDP	49	9045 → 9004	Len=7

i. ¿Cuál es la dirección IP que envía el primer datagrama?, ¿desde cuál puerto?

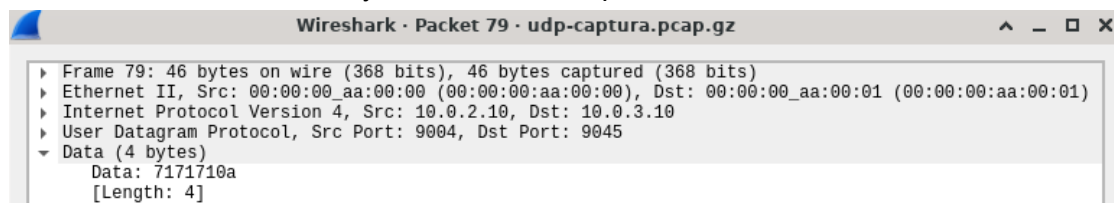
Dirección IP de origen: 10.0.2.10

Puerto de origen: 9004

ii. ¿Cuántos datos se envían en un sentido y en el otro?

En el sentido de ida (Fila 79: 10.0.2.10 → 10.0.3.10):

Se envían exactamente 4 bytes de datos de aplicación.



En el sentido de vuelta / respuesta (Fila 80: 10.0.3.10 → 10.0.2.10):

Se envían exactamente 7 bytes de datos de aplicación

